

Timer y PWM con NUCLEO-F446RE

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Introducción a los Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

Resumen de la clase:

- 3.1 Temas para la semana 4.
- 3.2 Repaso: TIMER.
- 3.3 Configuración TIMER con CUBE MX.
- 3.4 Uso de las herramientas HAL.
- 3.5 Repaso: PWM.
- 3.6 Configuración TIMER con CUBE MX.
- 3.7 Uso de las herramientas HAL.

Temas para la semana 4

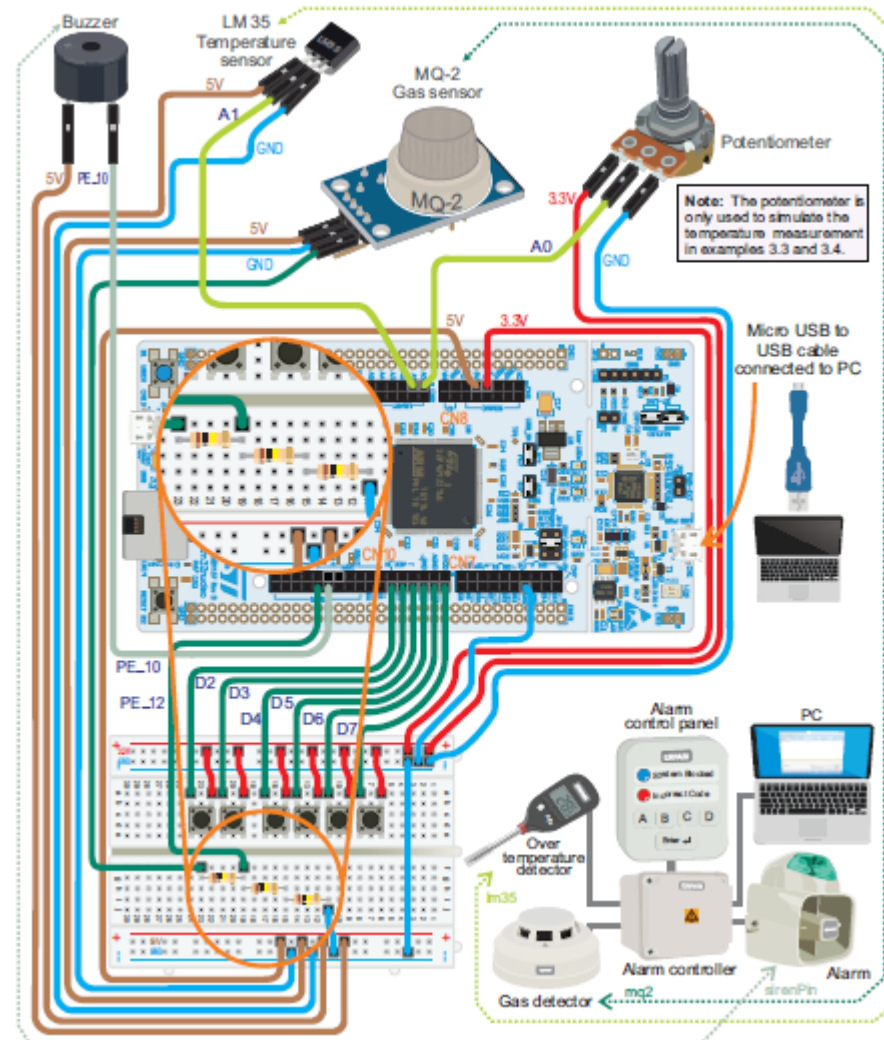


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.1 Temas para la semana 3.

3	Contenido de las clases de la semana 3. Teoría: Fundamentos de ADC. Uso de sensores analógicos y potenciómetros. Práctica: Lectura de señales analógicas. Creación de medidores y gráficos en tiempo real para representar los valores del ADC enviados por la placa NUCLEO-F446RE.	Será informada por el docente a cargo.	1, 2, 3 y 4.
4	Contenido de las clases de la semana 4. Teoría: Temporizadores (Timers) y sus registros de control. Interrupciones externas. Concepto de frecuencia. PWM. Práctica: Generación de señales PWM. Implementación de retardos y generación de tiempos periódicos. Uso de botones por interrupción. Interfaz Node-RED y placa NUCLEO-F446RE, haciendo uso de interrupciones externas.	Será informada por el docente a cargo.	1, 2, 3 y 4.

Repaso: Timer y PWM

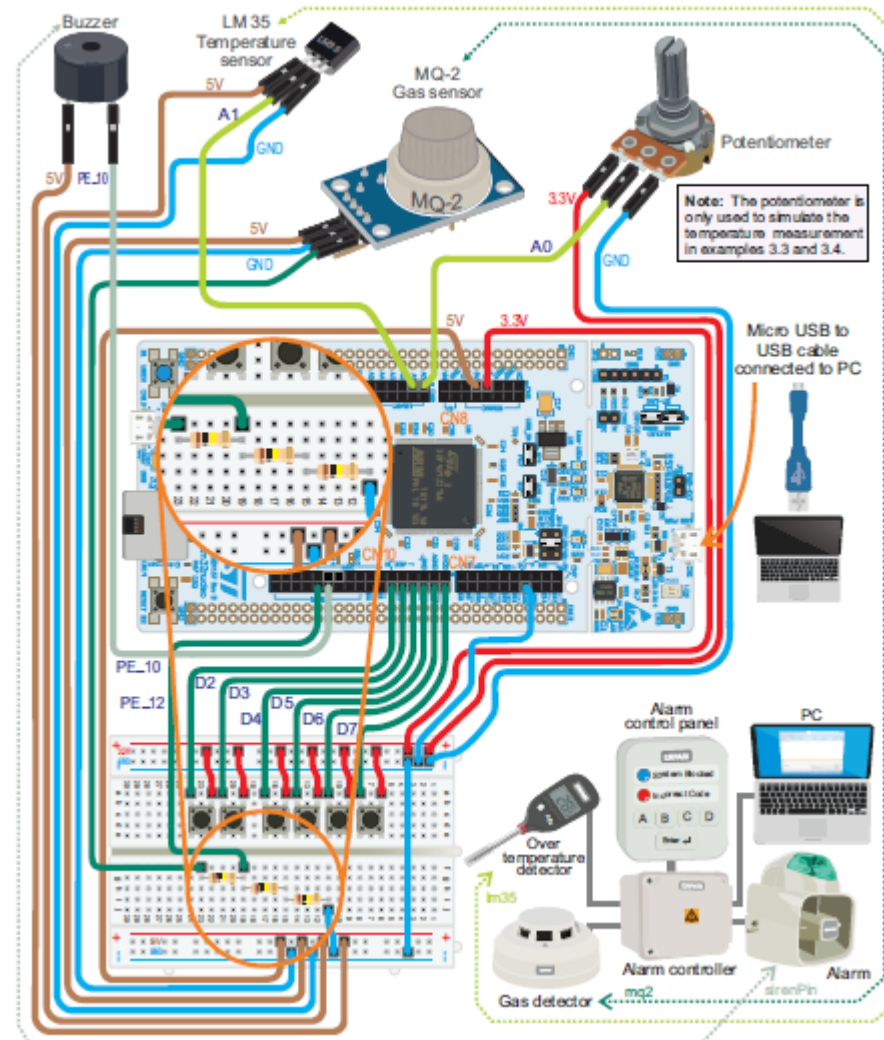


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.2 Repaso: Timer y PWM.

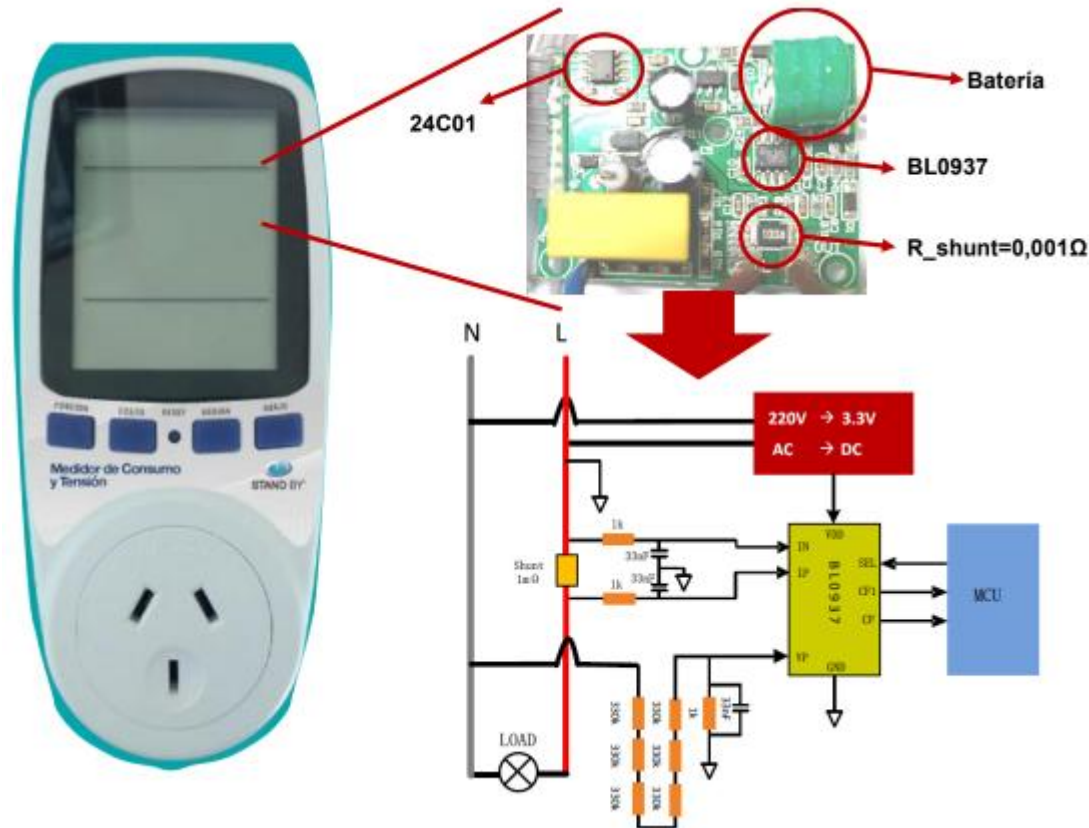
- **Conceptos básicos del temporizador:** Un timer es un componente de hardware especial que se encuentra dentro del microcontrolador.
- Su función: cuenta, por ejemplo un temporizador de 8 bits cuenta de 0 a 255.
- La mayoría de los temporizadores se reinician al alcanzar su valor máximo.

3.2 Repaso: Timer y PWM.

- Funciones donde se utilizan los temporizadores:
- **Comparación de salida (OC):** cambia el estado de un pin cuando un temporizador alcanza un valor determinado.
- **Captura de entrada (IC):** mide el número de conteos de un temporizador entre eventos en un pin.
- **Modulación por ancho de pulso (PWM):** señal donde se puede ajustar el tiempo de encendido y apagado (ciclo de trabajo).

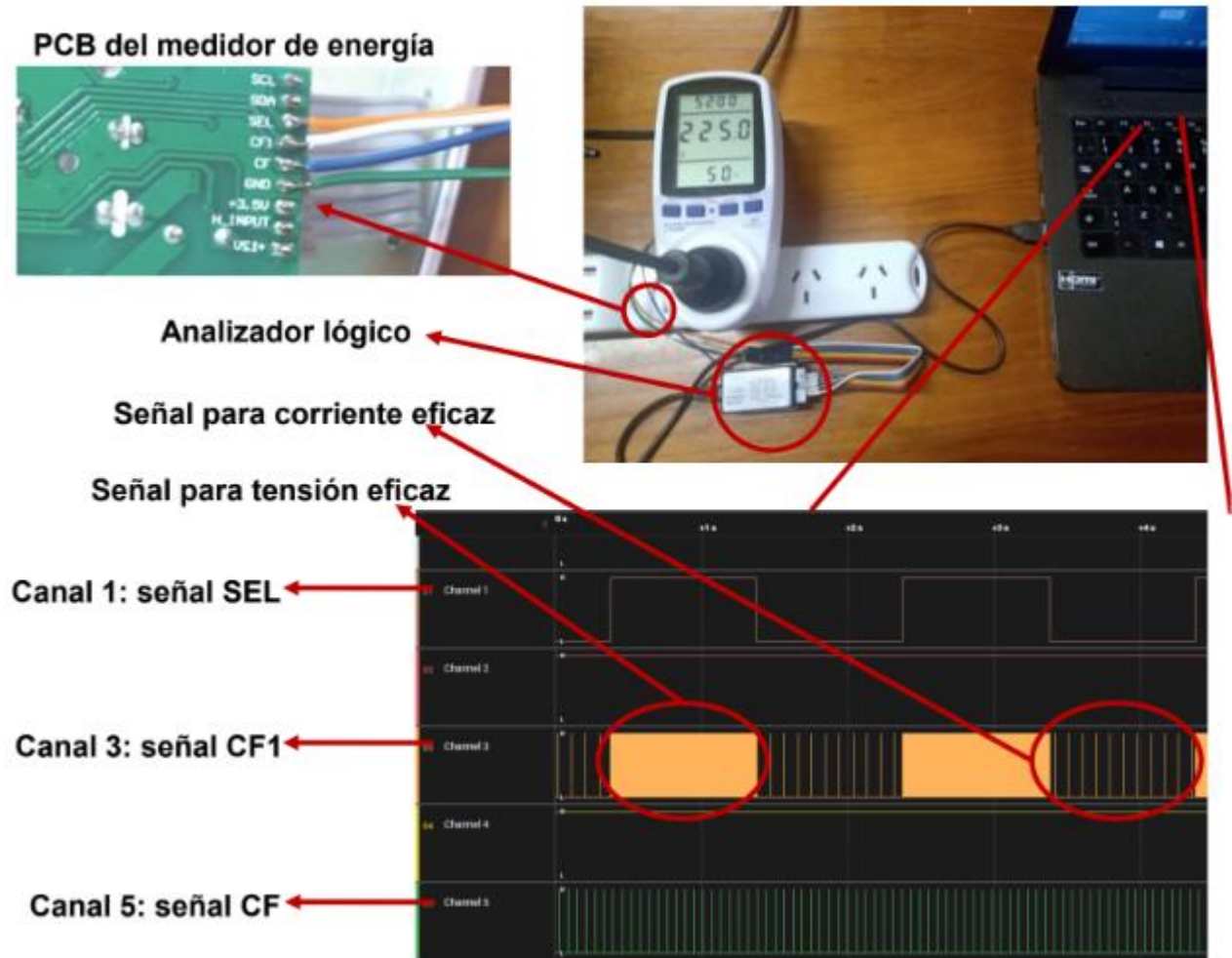
3.2 Repaso: Timer y PWM.

- Usos: Medir magnitudes eléctricas con el BL0937.



3.2 Repaso: Timer y PWM.

- Usos de interrupciones para medir magnitudes eléctricas.



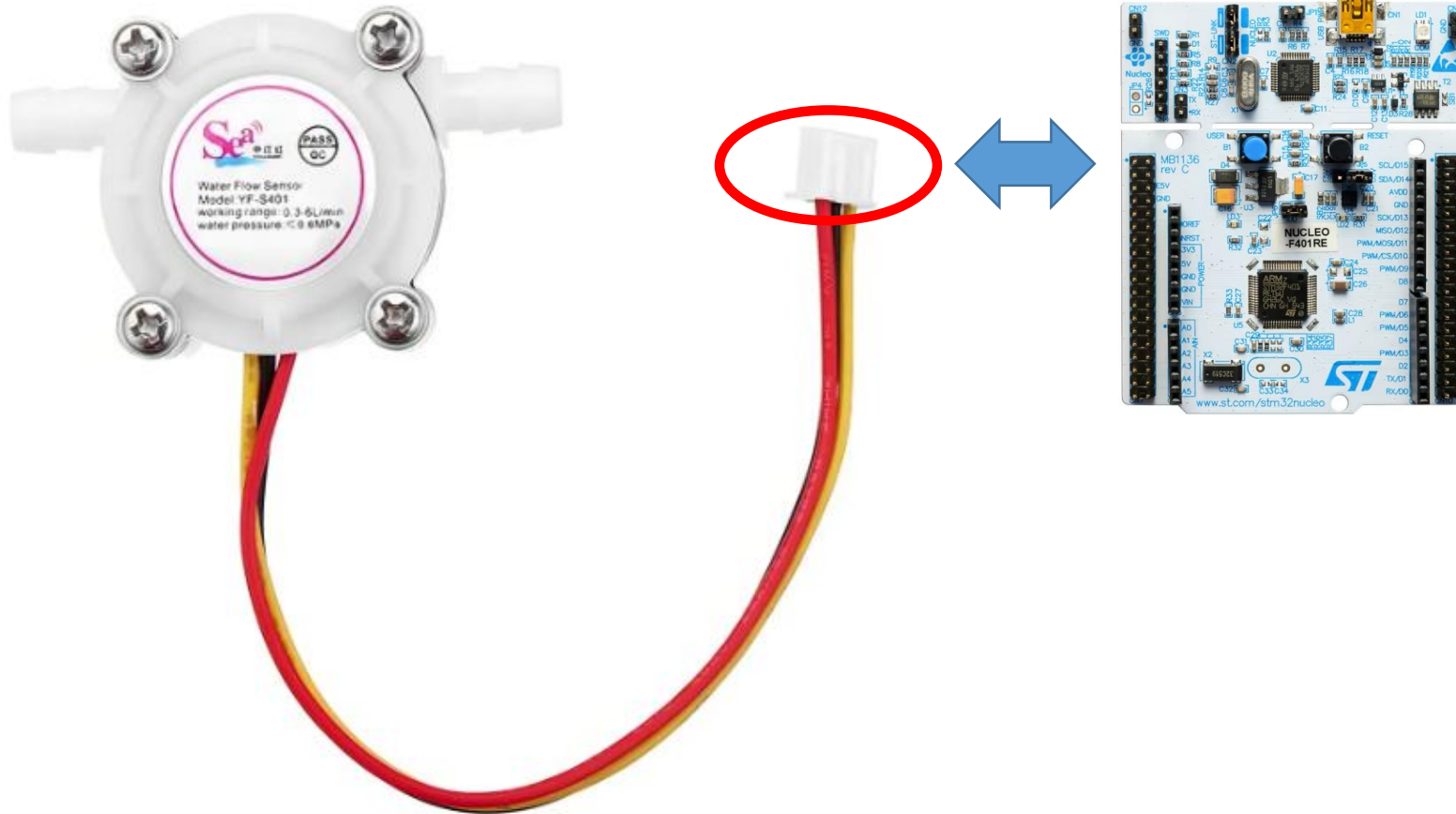
3.2 Repaso: Timer y PWM.

- Dispositivos con señal de salida pulsante.



3.2 Repaso: Timer y PWM.

- Dispositivos con señal de salida pulsante.



3.2 Repaso: Timer y PWM.

- Temporizadores de propósito general (TIM2 a TIM5).
- Constan de un **contador de recarga automática de 16 o 32 bits**, controlado por un prescaler (PSC) programable.
- La mayoría se conectan al reloj principal de la CPU y avanza (incrementa) cada vez que recibe un pulso de reloj.

3.2 Repaso: Timer y PWM.

- Un temporizador de 16 bits puede contar hasta 65535, si deseamos medir eventos de mayor duración necesitamos usar un PSC. Ej: un PSC de 80 convierte una señal de 80 MHz en una de 1 MHz.
- Los temporizadores son independientes y pueden sincronizarse.
- Características del TIM2 a TIM5: Contador de 16 bits (TIM3 y TIM4) o 32 bits (TIM2 y TIM5)
- PSC programable de 16 bits para dividir la frecuencia del reloj del contador por cualquier factor entre 1 y 65536.

3.2 Repaso: Timer y PWM.

- El STM32F446RE posee:
- HSI (High Speed Internal): Reloj interno de 16 MHz (oscilador RC).
- HSE (High Speed External): Reloj externo (cristal de cuarzo) de entre 4 y 26 MHz (más preciso que el HSI).
- LSE (Low Speed External): Cristal de 32.768 kHz para el reloj de tiempo real (RTC).
- PLL (Phase Locked Loop): Multiplicador de frecuencia, puede elevar el reloj del sistema (SYSCLK) hasta los 180 MHz.

3.2 Repaso: Timer y PWM.

- Distribución por Buses:
- AHB (Advanced High-performance Bus): Bus de alta velocidad conectado directamente al SYSCLK. Aquí se encuentran las GPIO.
- APB1 (Low Speed): Frecuencia máxima de 45 MHz y conecta periféricos como TIM2 y TIM3 a 90MHz.
- APB2 (High Speed): Frecuencia máxima de 90 MHz. Conecta periféricos como TIM1, ADC1, USART1.

3.2 Repaso: Timer y PWM.

- Un temporizador se vincula con:
- PSC: Divide la frecuencia de entrada para contar más lento.
- ARR (Auto-reload Register): Define el valor máximo del contador (el periodo).
- CNT (Counter): Valor actual de la cuenta.
- CCR (Capture/Compare Register): Se usa para definir el "Duty Cycle" en PWM o para capturar tiempos externos.

Configuración TIMER con CUBE MX

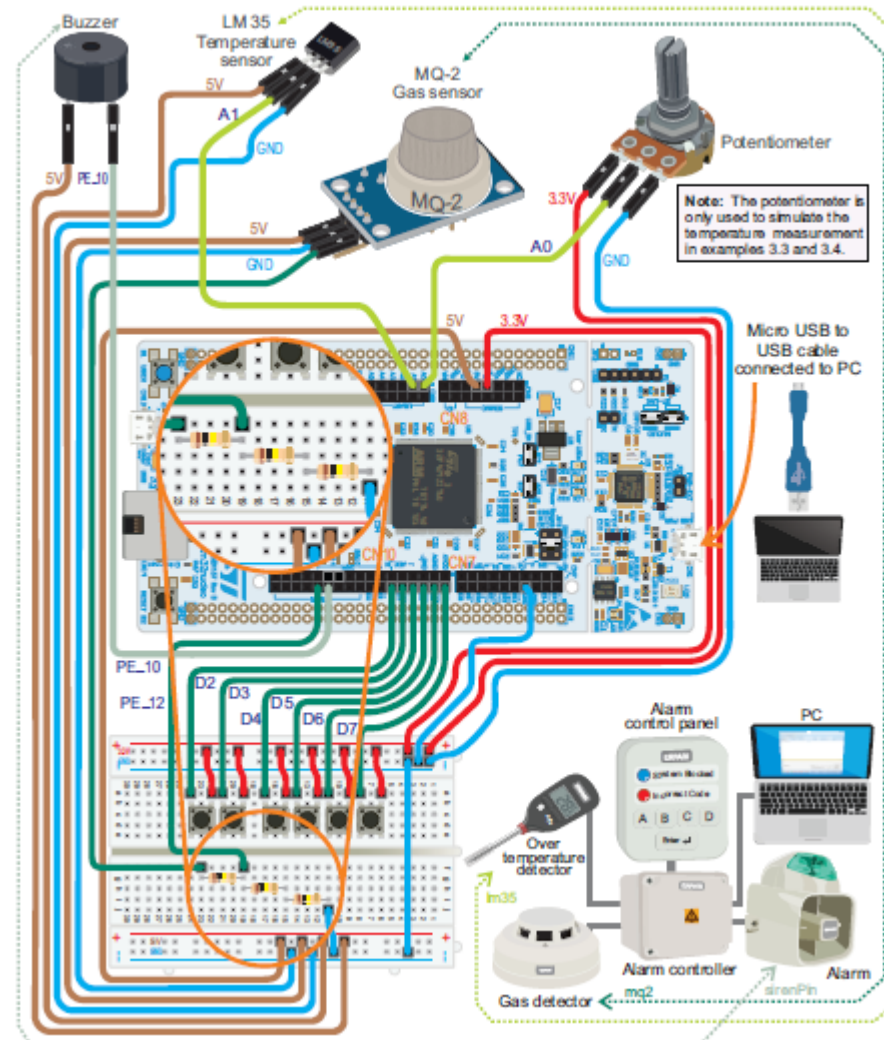


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.3 Configuración TIMER con CUBE MX

1. Ecuaciones y Registros Clave:

- Ecuación para la frecuencia de la interrupción (fint) y el periodo de tiempo (Tint), esta se calcula con las siguientes fórmulas:

$$fint = fTIM_CLK / [(PSC+1) \cdot (ARR+1)]$$

$$Tint = 1/fint = (PSC+1) \cdot (ARR+1) / fTIM_CLK$$

3.3 Configuración TIMER con CUBE MX

- fTIM_CLK: frecuencia del reloj que alimenta al Timer.
- Para el F446RE, el TIM2 está en el bus APB1.
- PSC: Divisor previo del reloj (16 bits: 0 a 65535).
- ARR (Auto-Reload Register): El valor hasta el cual cuenta el timer antes de reiniciarse y generar la interrupción.

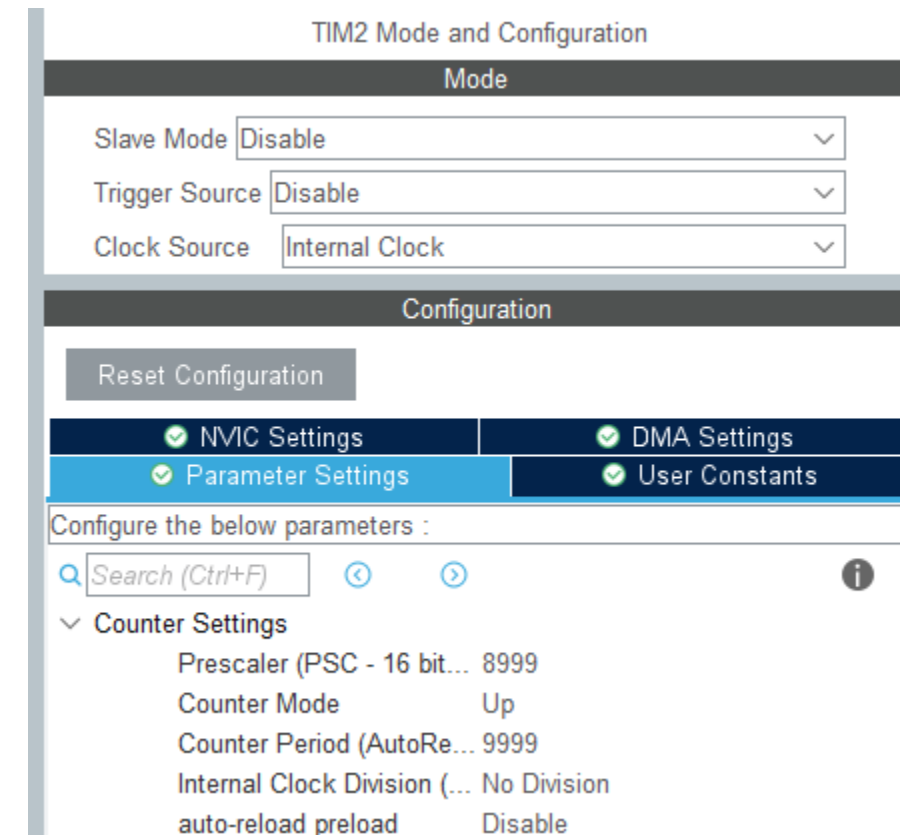
En TIM2 es de 32 bits: 0 a 4.294.967.295.

3.3 Configuración TIMER con CUBE MX

- Configurar el TIM2 para generar una señal de 1 Hz (1 segundo) vamos a colocar una frecuencia de 90 MHz en APB1:
- Clock Configuration: el casillero APB1 timer clocks (MHz) esté en 90.
- Pinout & Configuration -> Timers -> TIM2:
- Clock Source: Selecciónalo como Internal Clock.

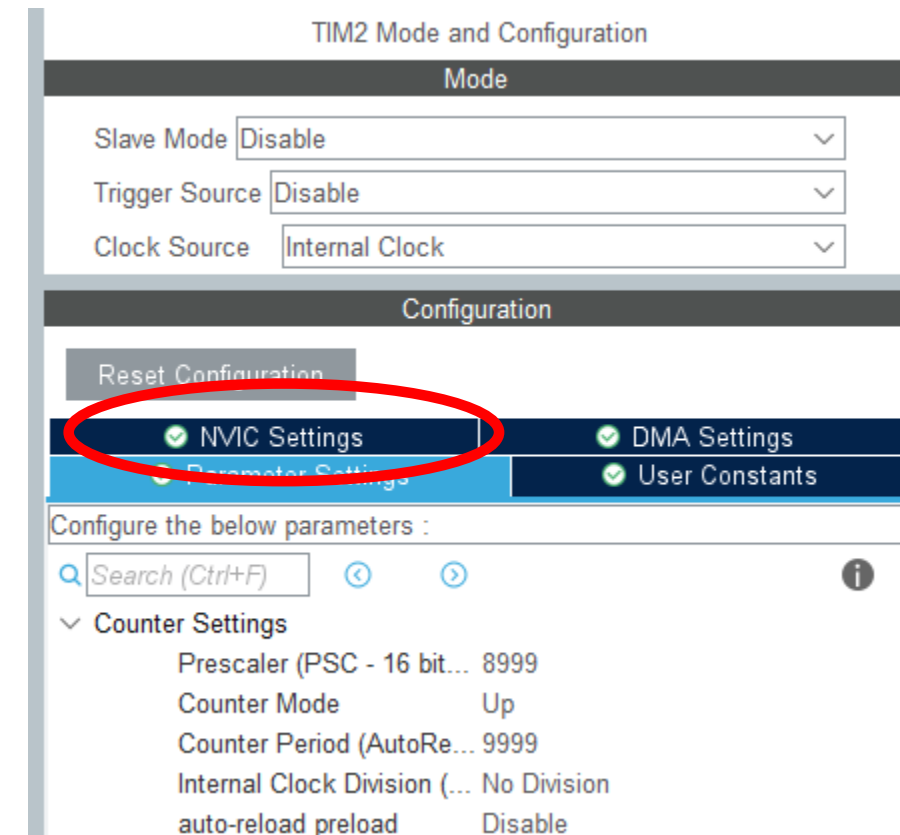
3.3 Configuración TIMER con CUBE MX

- Parameter Settings: $T_{int} = 1/f_{int} = (PSC+1) \cdot (ARR+1) / f_{TIM_CLK}$
- Proponemos un PSC: 8999, divide 90 MHz entre 9000, obteniendo 10 kHz.
- Counter Period (ARR): 9999, cuenta 10,000 pulsos, lo que equivale exactamente a 1 segundo.



3.3 Configuración TIMER con CUBE MX

- En NVIC Settings, activar el checklist de TIM2 global interrupt.
- Generar Código.



Uso de las herramientas HAL

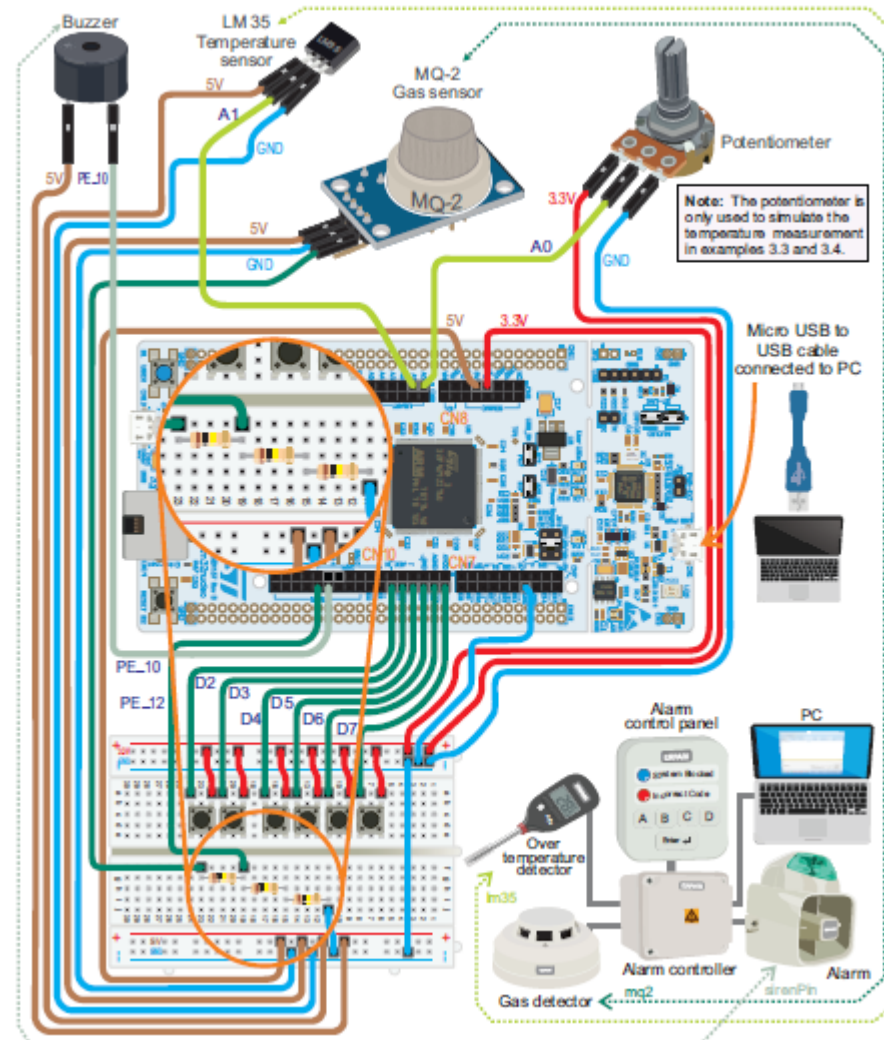


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.4 Uso de las Herramientas HAL

Prende y apaga un led cada segundo:

```
#include "main.h"
#include "tim.h"
#include "gpio.h"
void SystemClock_Config(void);
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM2_Init();
    HAL_TIM_Base_Start_IT(&htim2);
    while (1) {
    }
}
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    }
}
```


3.4 Uso de las Herramientas HAL

- `HAL_TIM_Base_Start_IT(&htim2);` función que inicia el timer, configura los registros internos.
- `void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim);`
Es una función callback, la librería HAL llama esta función cuando el temporizador completa un periodo.
- `if (htim->Instance == TIM2):` control, si el timer es el TIM2, ejecuta este código.

3.4 Uso de las Herramientas HAL

Prende y apaga un led cada segundo en función del botón de usuario:

```
#include "main.h"
#include "tim.h"
#include "gpio.h"
void SystemClock_Config(void);
int main(void){
    uint8_t parpadeo_activo = 0;
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM2_Init();
```

```
    while (1){
        if (HAL_GPIO_ReadPin(boton_GPIO_Port, boton_Pin) == GPIO_PIN_RESET){
            HAL_Delay(20);
            while (HAL_GPIO_ReadPin(boton_GPIO_Port, boton_Pin) ==
                GPIO_PIN_RESET){
            }
            HAL_Delay(20);
            if (parpadeo_activo == 0){
                parpadeo_activo = 1;
                HAL_TIM_Base_Start_IT(&htim2);
            }else{
                parpadeo_activo = 0;
                HAL_TIM_Base_Stop_IT(&htim2);
                HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
            }
        }
    }
}
```

3.4 Uso de las Herramientas HAL

- Iniciar y suspender la interrupción:
- Se elimina el HAL_TIM_Base_Start_IT al inicio (sistema en reposo).
- Se puede condicionar el inicio (HAL_TIM_Base_Start_IT) al presionar un botón.
- Control del Periférico: Usamos HAL_TIM_Base_Stop_IT(&htim2) para congelar el conteo del hardware.
- Al detenerlo, el micro no gasta recursos.

PWM

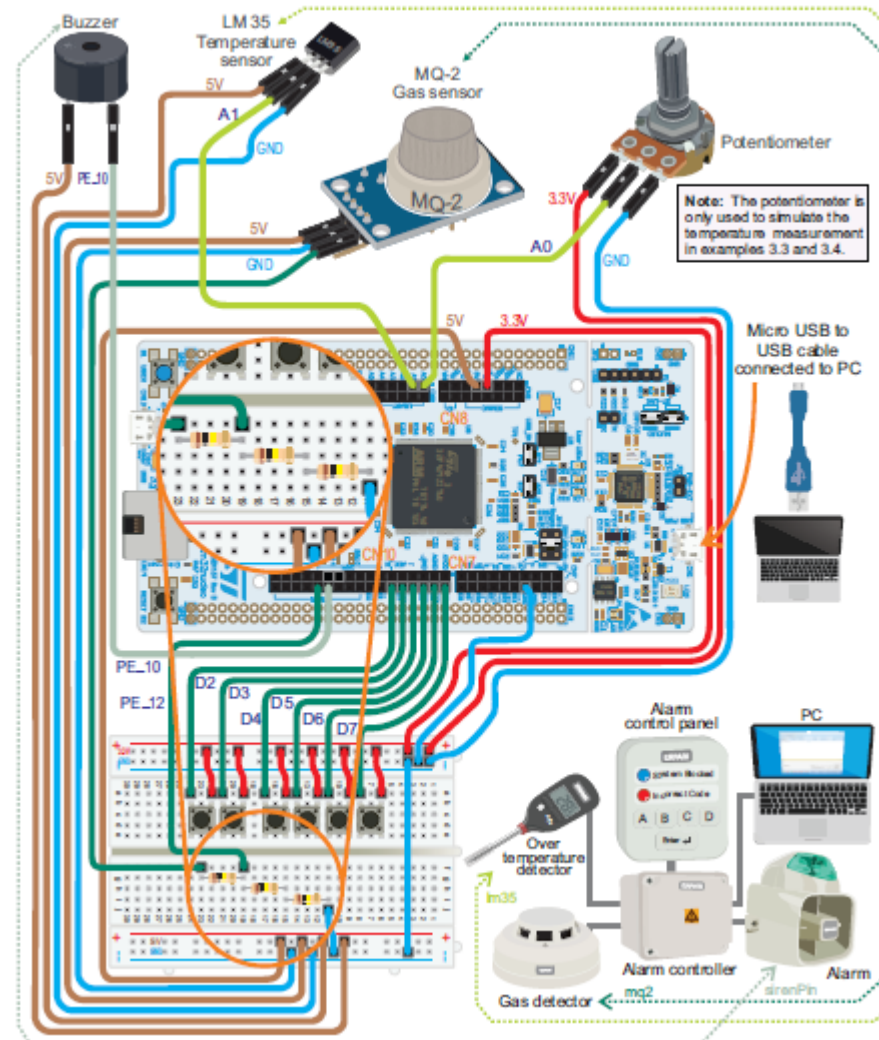


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.
<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.5 Repaso: PWM.

- En lugar de usar el Timer solo como un contador, configuramos el hardware para que compare el valor del contador (CNT) con un registro de captura/comparación (CCR).
- Cuando el contador coincide con ese valor, el hardware cambia automáticamente el estado de un pin (de 0 a 1 o viceversa), sin intervención del CPU.1.

Configuración PWM con CUBE MX

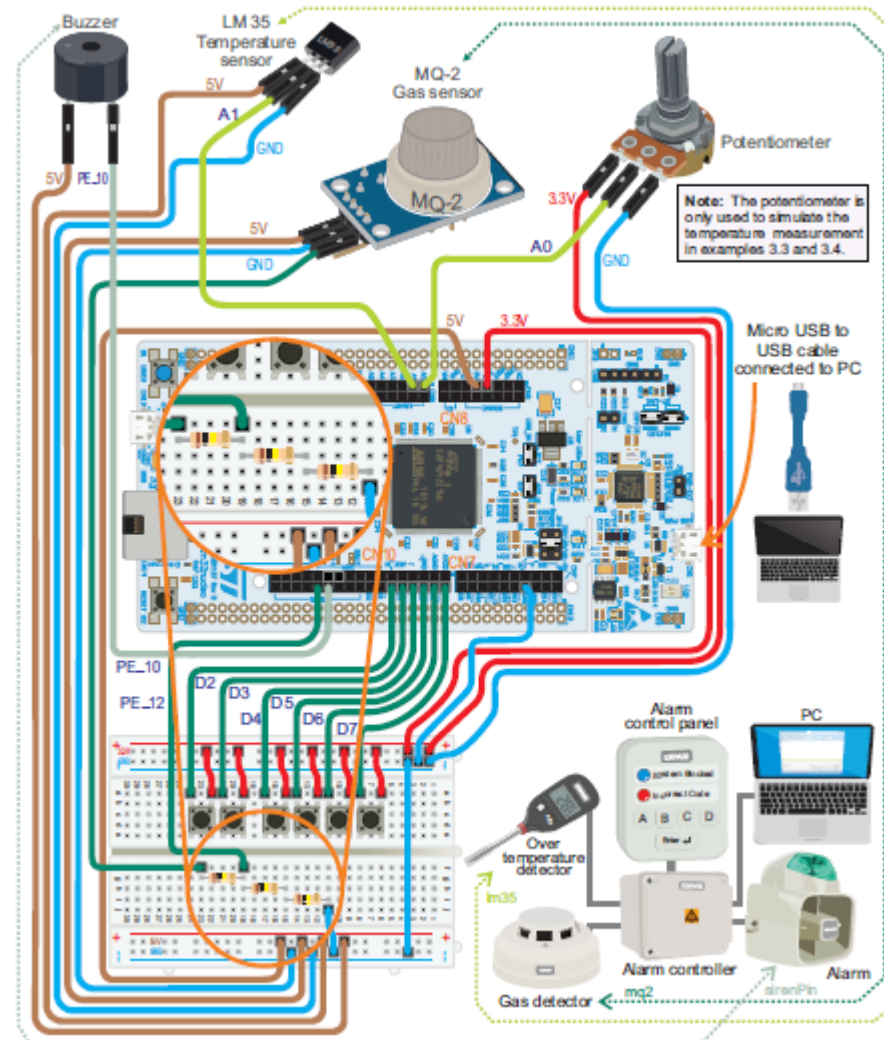


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.6 Configuración PWM con CUBE MX

- La frecuencia y el ciclo de trabajo (duty cycle) del PWM se controlan así:
- Frecuencia: Depende del PSC (Prescaler) y el ARR (Periodo/Auto-Reload).

$$f_{pwm} = f_{TIM_CLK} / [(PSC+1) \cdot (ARR+1)]$$

$$T_{int} = 1/f_{pwm} = (PSC+1) \cdot (ARR+1) / f_{TIM_CLK}$$

3.6 Configuración PWM con CUBE MX

- Ciclo de trabajo (Duty Cycle): Depende del registro CCR (Capture Compare Register).
- $\text{Duty} = \text{CCR} \cdot 100\% / (\text{ARR} + 1)$

3.6 Configuración PWM con CUBE MX

- Configuración:
- Timers -> TIM3.
- Mode: Cambia de Disable a PWM Generation CH1.
- Este corresponde al pin PA6.
- Como el timer:
- Parameter Settings: Prescaler (PSC): con un clk a 90 MHz y quieres que el contador avance a 1KHz, PSC=8999.

3.6 Configuración PWM con CUBE MX

- Counter Period (ARR): Define la resolución. Si pones 9999, tu periodo será de 1s.
- PWM Generation CH1:Mode: PWM Mode 1 (es el estándar).
- Pulse: Es el valor inicial del CCR.

Uso de las herramientas HAL

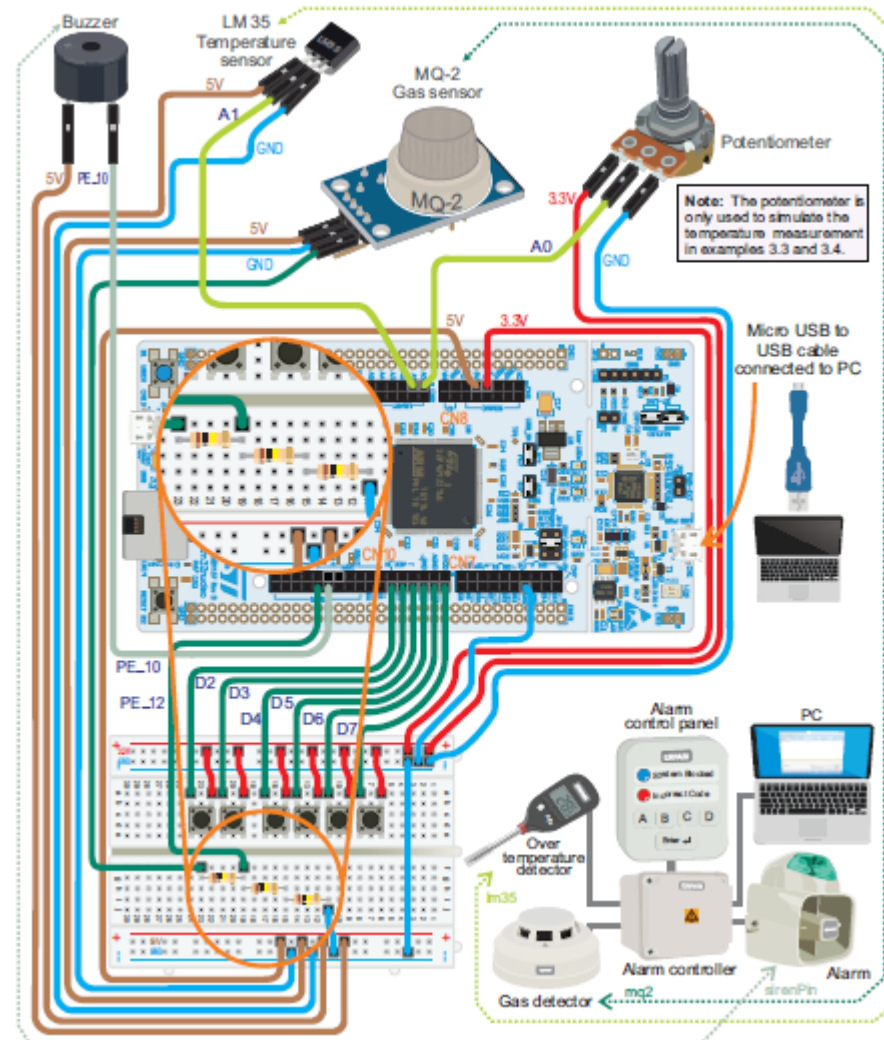
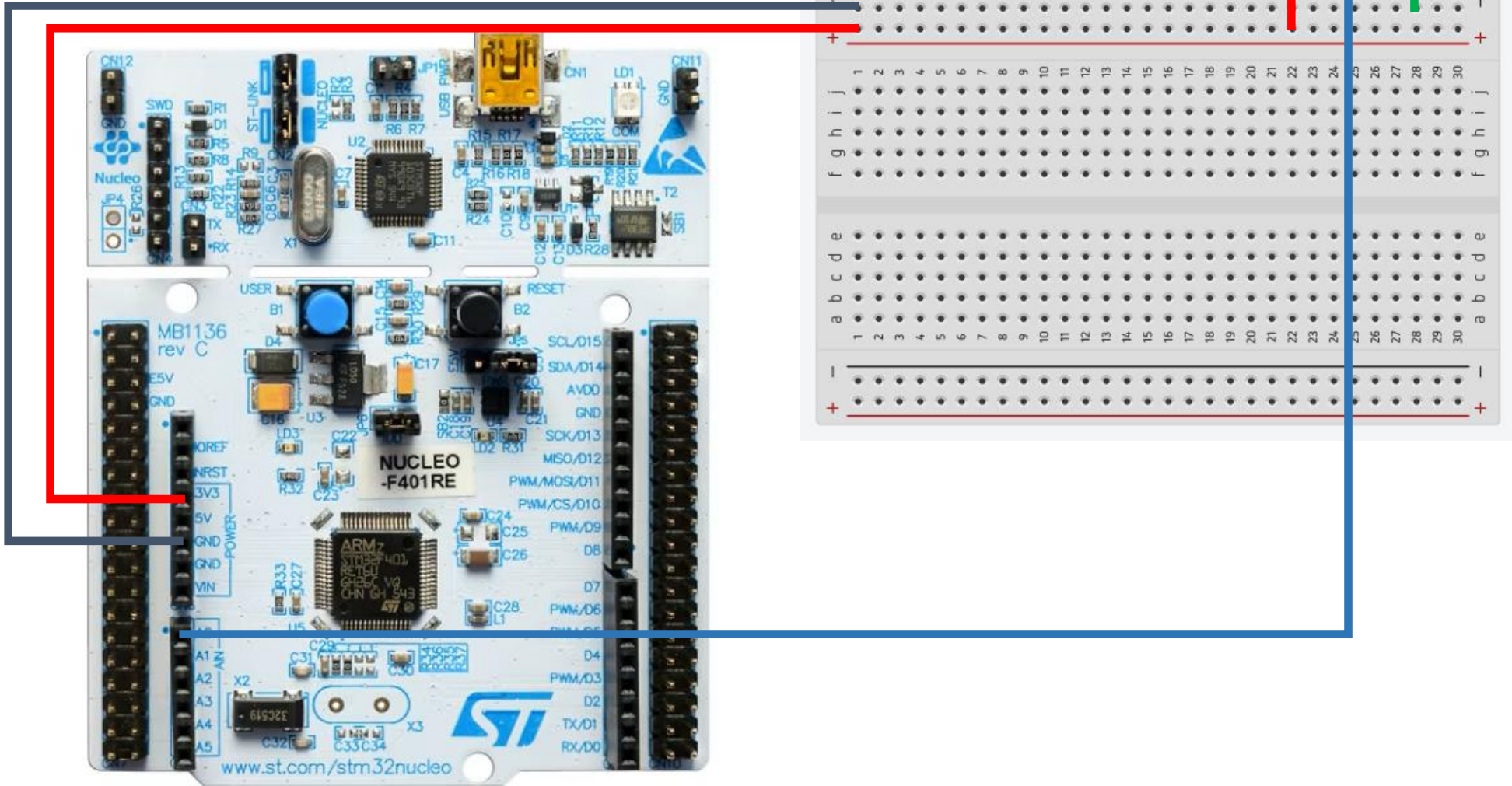


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.7 Uso de las Herramientas HAL



3.7 Uso de las Herramientas HAL

```
#include "main.h"
#include "adc.h"
#include "tim.h"
#include "gpio.h"
```

```
while (1){
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){
        valor_adc = HAL_ADC_GetValue(&hadc1);
        duty_scalado = (valor_adc * 9999) / 4095;
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty_scalado);
    }
}
```

```
void SystemClock_Config(void);
int main(void){
    uint32_t duty_scalado;
    uint32_t valor_adc;
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_TIM3_Init();
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
```

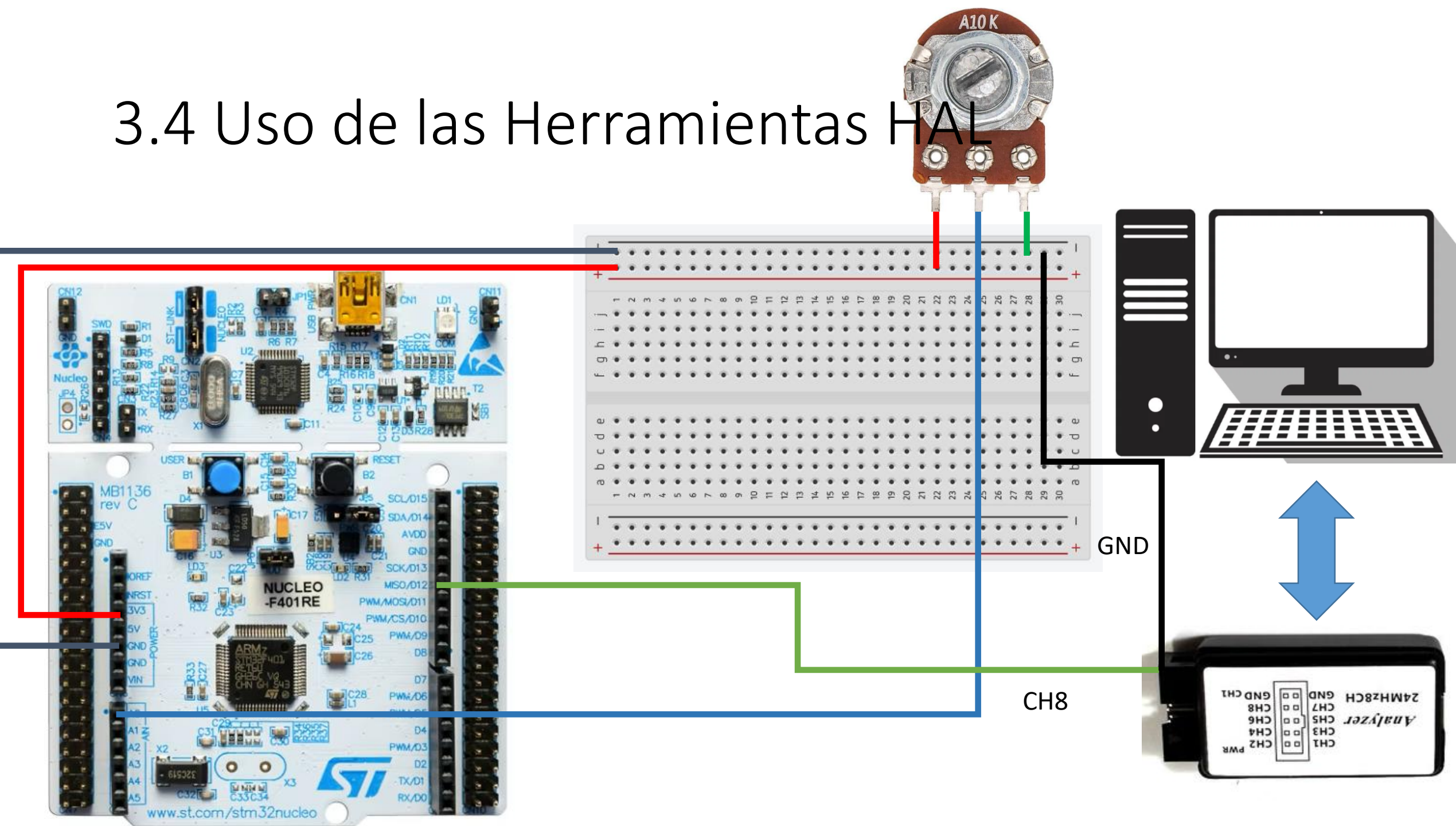
3.7 Uso de las Herramientas HAL

- `MX_GPIO_Init()`, `MX_ADC1_Init()`, `MX_TIM3_Init()`: funciones auto-generadas por CubeMX que escriben los registros de configuración.
- `HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1)`: inicia el comparador del canal 1 del Timer 3, para que el pin físico (PA6) empiece a generar los pulsos PWM en función del registro CCR.

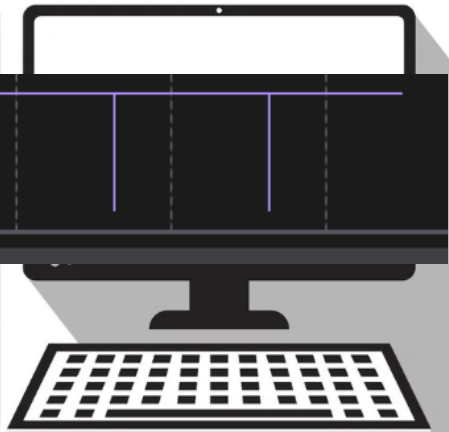
3.7 Uso de las Herramientas HAL

- `uint32_t duty_scalado = (valor_adc * 9999) / 4095`: convierte el mundo analógico (ADC 12-bits) a un numero acorde al ARR (9999).
- `__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty_scalado)`: el compilador reemplaza esto directamente por una escritura en el registro CCR del hardware.

3.4 Uso de las Herramientas HAL



A close-up photograph of a red terminal block labeled 'A10 K'. It features a large central screw terminal and three smaller screw terminals below it. Three wires are plugged into the bottom terminals: a red wire on the left, a blue wire in the middle, and a green wire on the right. The word 'HAL' is superimposed in large black letters across the center of the image.



Bibliografia.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media